K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 05 TASKS

TASK 01:

```
#include<iostream>
using namespace std;

class Stack
{
  int arr[6];
  int top;

  public:
  Stack()
  {
    top=-1;
  }

  void push(int value)
  {
    if(top==5)
    {
      cout<<"Stack overflow.";
      return;
    }

    arr[++top]=value;

  }

  void pop()
  {
    if(top==-1)
    {
      cout<<"Stack underflow.";
      return;
    }

    top--;
  }

  void peek()
  {
    if(top==-1)
```

DSA LAB 05 TASKS

```
    {
      cout<<"Stack is empty ."<<endl;
    }

    cout<<"Current top : "<< arr[top]<<endl;
  }

  bool isempty()
  {
    return top==-1;
  }

  void display()
  {
    for(int i=top;i>=0;i--)
    {
      cout<<arr[i]<<" ";

    }

    cout<<endl;
  }
};

int main()
{
  Stack st;

  st.push(12);
  st.push(25);
  st.push(17);
  st.push(31);
  st.push(44);
  st.push(19);

  st.pop();
  st.peek();

  st.pop();
  st.peek();

  st.pop();
```

DSA LAB 05 TASKS

```
  st.peek();

  st.push(52);
  st.peek();

  st.pop();
  st.peek();

  st.pop();
  st.peek();

  st.display();

  return 0;
}
```

```
Problems  Output  Debug Console  Terminal  Ports  Spell Checker        Code

Current top : 44
Current top : 31
Current top : 17
Current top : 52
Current top : 17
Current top : 25
25 12
PS C:\Users\Sumaiya Abubakar\OneDrive - FAST National University\Documents\3rd Semester\DSA LAB\LAB TASK 05>
```

TASK 02:

```
#include <iostream>
#include <string>
using namespace std;

class Node

{
public:
  string data;
  Node *next;

  Node(string word)
  {
    data = word;
    next = nullptr;
  }
};
```

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 05 TASKS

```
class Stack
{
   Node *top;

public:
   Stack()
   {
      top = nullptr;
   }

   void push(string word)
   {
      Node *newNode = new Node(word);

      newNode->next = top;
      top = newNode;
   }

   void pop()
   {
      if (top == nullptr)
      {
         return;
      }

      Node *temp = top;
      top = top->next;
      delete temp;
   }

   string peek()
   {
      if (top == nullptr)
      {
         return "";
      }

      return top->data;
   }

   bool isEmpty()
```

DSA LAB 05 TASKS

```
  {
    return top == nullptr;
  }
};

class TextEditor
{
  string document;

  Stack undoStack;
  Stack redoStack;

public:
  void type(string word)
  {
    if (document.empty())
    {
      document = word;
    }
    else
    {
      document = document + " " + word;
    }

    undoStack.push(word);

    while (!redoStack.isEmpty())
    {
      redoStack.pop();
    }
  }

  void undo()
  {
    if (undoStack.isEmpty())
    {
      return;
    }

    string word = undoStack.peek();
    undoStack.pop();
```

DSA LAB 05 TASKS

```
    if (document == word)
    {
        document = "";
    }
    else
    {
        document.erase(document.length() - word.length() - 1);
    }

    redoStack.push(word);
}

void redo()
{
    if (redoStack.isEmpty())
    {
        return;
    }

    string word = redoStack.peek();
    redoStack.pop();

    if (document.empty())
    {
        document = word;
    }
    else
    {
        document = document + " " + word;
    }

    undoStack.push(word);
}

void print()
{
    cout << document << endl;
}
};

int main()
{
```

DSA LAB 05 TASKS

```
    TextEditor editor;

    editor.type("Hello");
    editor.print();

    editor.type("World");
    editor.print();

    editor.undo();
    editor.print();

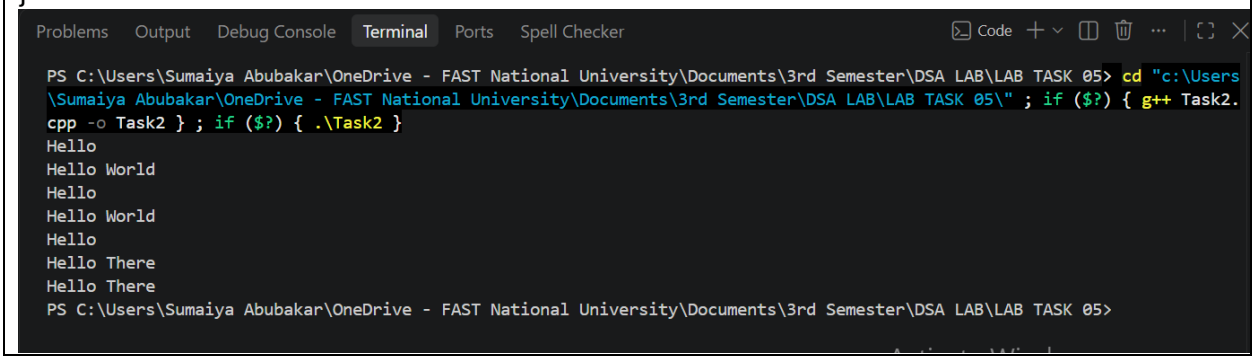
    editor.redo();
    editor.print();

    editor.undo();
    editor.print();

    editor.type("There");
    editor.print();

    editor.redo();
    editor.print();

    return 0;
}
```



TASK 03:

```
#include <iostream>
using namespace std;

class Node
{
```

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 05 TASKS

```
public:
  int job;
  Node *next;

  Node(int j)
  {
    job = j;
    next = nullptr;
  }
};

class Stack
{
  Node *top;

public:
  Stack()
  {
    top = nullptr;
  }

  void push(int j)
  {
    Node *newNode = new Node(j);

    newNode->next = top;
    top = newNode;
  }

  void pop()
  {
    if (top == nullptr)
    {
      return;
    }

    Node *temp = top;
    top = top->next;
    delete temp;
  }

  int peek()
```

DSA LAB 05 TASKS

```
  {
    if (top == nullptr)
    {
      return -1;
    }

    return top->job;
  }

  bool isEmpty()
  {
    return top == nullptr;
  }
};

class Queue
{
  Node *front;
  Node *rear;

public:
  Queue()
  {
    front = rear = nullptr;
  }

  bool isEmpty()
  {
    return front == nullptr;
  }

  void enqueue(int j)
  {
    Node *newNode = new Node(j);

    if (front == nullptr)
    {
      front = rear = newNode;
      return;
    }

    rear->next = newNode;
```

DSA LAB 05 TASKS

```
    rear = newNode;
}

void dequeue()
{
    if (isEmpty())
    {
        cout << "\nQueue is empty.";
        return;
    }

    Node *temp = front;
    front = front->next;

    if (front == nullptr)
    {
        rear = nullptr;
    }

    delete temp;
}

int peek()
{
    if (isEmpty())
    {
        cout << "\nQueue is empty. ";
        return -1;
    }

    return front->job;
}

void display()
{
    if (isEmpty())
    {
        cout << "\nQueue is empty. ";
        return;
    }

    Node *temp = front;
```

DSA LAB 05 TASKS

```
    while (temp != nullptr)
    {
        cout << temp->job << " ";
        temp = temp->next;
    }

    cout << endl;
}

int countsize()
{
    int count = 0;

    Node *temp = front;

    while (temp != nullptr)
    {
        count++;
        temp = temp->next;
    }

    return count;
}
};

void reverseFirstK(Queue &q, int k)
{
    Stack st;
    for (int i = 0; i < k; i++)
    {
        int temp1 = q.peek();
        q.dequeue();
        st.push(temp1);
    }

    for (int i = 0; i < k; i++)
    {
        int temp2 = st.peek();
        st.pop();
        q.enqueue(temp2);
    }
```

DSA LAB 05 TASKS

```
    int remaining = q.countsize() - k;

    for (int i = 0; i < remaining; i++)
    {
        int temp3 = q.peek();
        q.dequeue();
        q.enqueue(temp3);
    }
}

int main()
{
    Queue q;

    for(int i = 1; i <= 7; i++)
    {
        q.enqueue(i);
    }

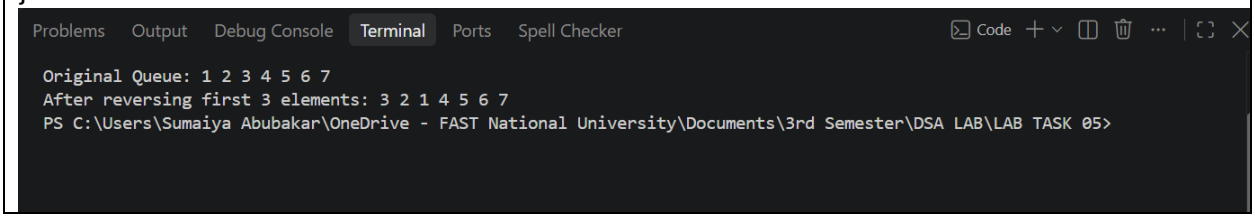
    cout << "Original Queue: ";
    q.display();

    int k = 3;

    reverseFirstK(q, k);

    cout << "After reversing first " << k << " elements: ";
    q.display();

    return 0;
}
```



TASK 04:

```
#include<iostream>
using namespace std;
```

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 05 TASKS

```
class Queue
{
   int gateCapacity[6];
   int front;
   int rear;
   int count;

   public:
   Queue()
   {
      front=0;
      rear=-1;
      count=0;
   }

   bool isEmpty()
   {
      return count==0;
   }

   bool isFull()
   {
      return count==6;
   }

   void enqueue(int p_ID)
   {
      if(isFull())
      {
         cout<<"\nQueue overflow.";
         return;
      }

      rear=(rear+1)%6;
      gateCapacity[rear]=p_ID;

      count++;
   }

   int dequeue()
```

DSA LAB 05 TASKS

```
{
   if(isEmpty())
   {
      cout<<"\nQueue underflow. ";
      return -1;
   }

   int temp=gateCapacity[front];

   front=(front+1)%6;

   count--;

   return temp;
}

void peek()
{
   if(isEmpty())
   {
      cout<<"\nQueue is empty. ";
      return;
   }

   cout<<"\nFront passenger ID : "<<gateCapacity[front];
}

void display()
{
   if(isEmpty())
   {
      cout<<"\nQueue is empty. ";
      return;
   }

   int index = front;

   for(int i = 0; i < count; i++)
   {
      cout << gateCapacity[index] << " ";
      index = (index + 1) % 6;
   }
```

DSA LAB 05 TASKS

```
        cout<<endl;
    }
};

int main()
{
    Queue q;
    for(int i = 101; i <= 106; i++)
    {
        q.enqueue(i);
    }

    for(int i = 0; i < 3; i++)
    {
        cout << "Boarded: " << q.dequeue() << endl;
    }

    for(int i = 107; i <= 109; i++)
    {
        q.enqueue(i);
    }

    for(int i = 0; i < 2; i++)
    {
        cout << "Boarded: " << q.dequeue() << endl;
    }

    q.enqueue(110);

    cout << "Boarded: " << q.dequeue() << endl;

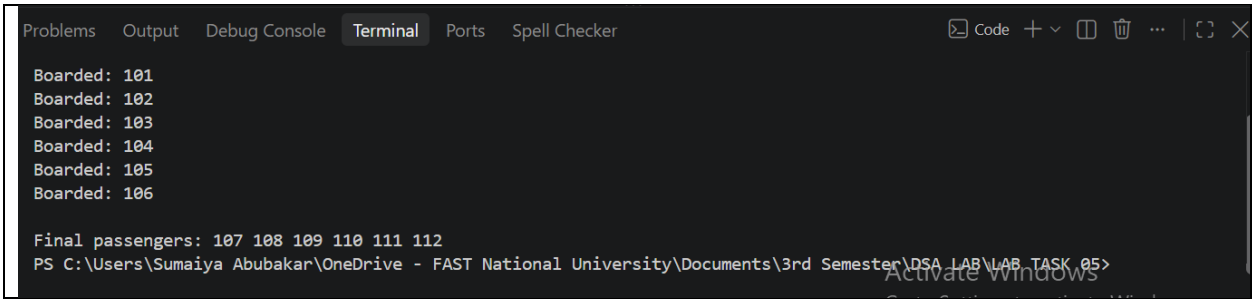
    for(int i = 111; i <= 112; i++)
    {
        q.enqueue(i);
    }

    cout << "\nFinal passengers: ";
    q.display();

    return 0;
}
```

K250512 SUMAIYA ABUBAKAR
9/27/26

DSA LAB 05 TASKS



TASK 05:

```
#include <iostream>
using namespace std;

class Node
{
public:
  char ch;
  Node *next;

  Node(char c)
  {
    ch = c;
    next = nullptr;
  }
};

class Stack
{
  Node *top;

public:
  Stack()
  {
    top = nullptr;
  }

  void push(char c)
  {
    Node *newNode = new Node(c);

    newNode->next = top;
```

DSA LAB 05 TASKS

```
    top = newNode;
}

void pop()
{
    if (top == nullptr)
    {
        return;
    }

    Node *temp = top;
    top = top->next;
    delete temp;
}

int peek()
{
    if (top == nullptr)
    {
        return -1;
    }

    return top->ch;
}

bool isEmpty()
{
    return top == nullptr;
}

int size()
{
    int count = 0;

    Node *temp = top;

    while (temp != nullptr)
    {
        count++;
        temp = temp->next;
    }
```

DSA LAB 05 TASKS

```
    return count;
  }
};

class MyQueue
{
  Stack incomingStack;
  Stack outgoingStack;

public:
  void enqueue(char c)
  {
    incomingStack.push(c);
  }

  char dequeue()
  {
    if (outgoingStack.isEmpty())
    {
      while (!incomingStack.isEmpty())
      {
        char temp = incomingStack.peek();
        incomingStack.pop();
        outgoingStack.push(temp);
      }
    }

    char temp = outgoingStack.peek();
    outgoingStack.pop();

    return temp;
  }
};

int main()
{
  MyQueue q;

  q.enqueue('A');
  q.enqueue('B');

  cout << q.dequeue() << endl;
```

DSA LAB 05 TASKS

```
    q.enqueue('C');

    cout << q.dequeue() << endl;
    cout << q.dequeue() << endl;

    return 0;
}
```

